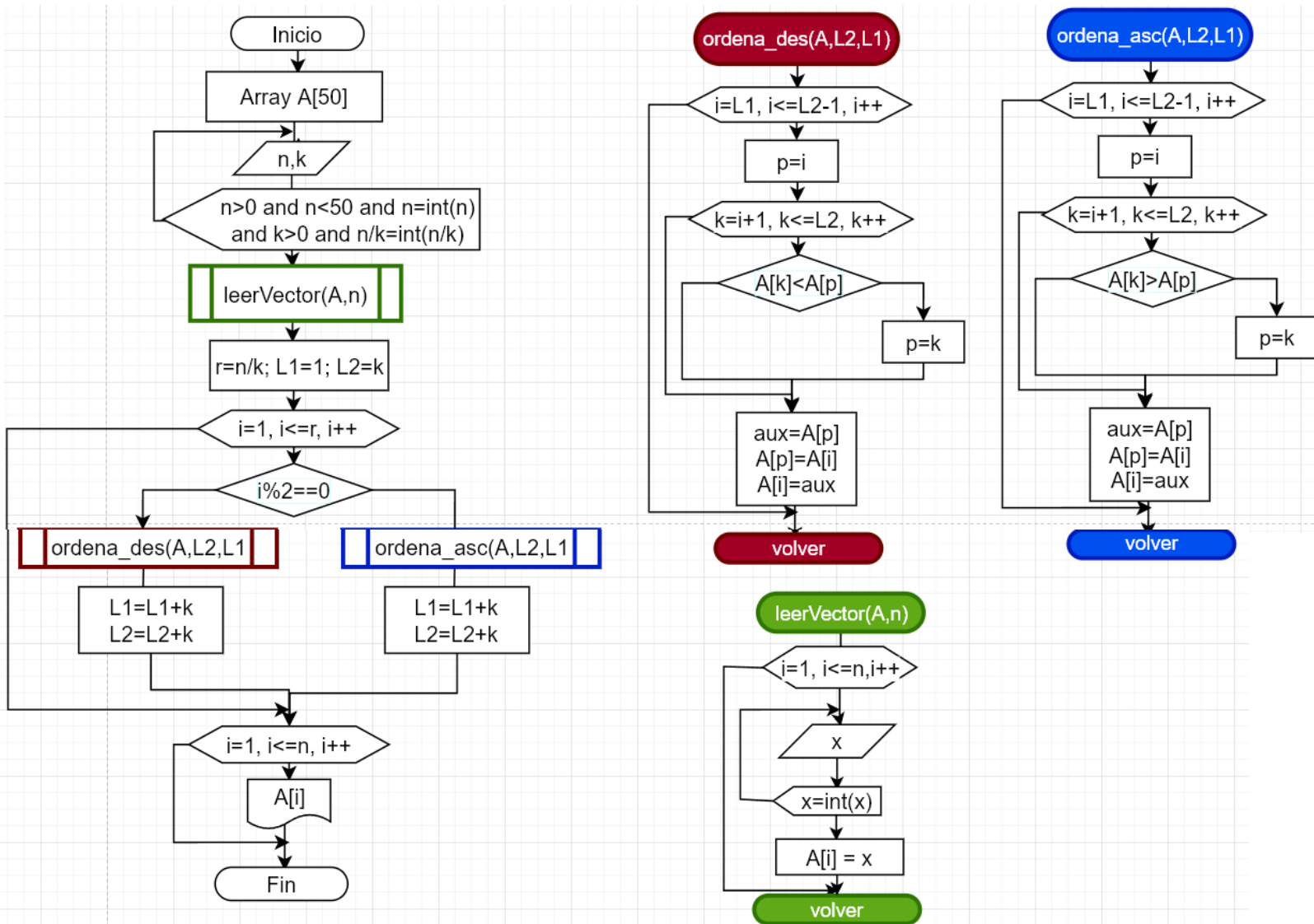


P18: DIVISIÓN DE VECTORES

P18: Alex Choque V.

C.I.: 13183020

1.- Dado un vector V que contiene n números, dividido en r sectores de k elementos cada uno. Los sectores pares ordenar en forma ascendente y los sectores impares ordenar en forma descendente. Leer k que divide exactamente a n:



```

//Ejercicio 1 division de vectores + - +
Scanner leer = new Scanner(System.in);
int A[]=new int[50];
int n;
int k;
do {
    n=leer.nextInt();
}while(n<=0 || n>50);
do {
    k=leer.nextInt();
}while(k<=0 || (n%k)!=0);
System.out.println("Inicio del programa: ");
leerVector(A,n);
int r=n/k; int L1=1; int L2=k;
for(int i=1; i<=r; i++) {
    if(i%2==0) {
        ordena_asc(A, L2, L1);
        L1=L1+k;
        L2=L2+k;
    }else {
        ordena_des(A, L2, L1);
        L1=L1+k;
        L2=L2+k;
    }
}
for(int i=1; i<=n;i++)
    System.out.print(A[i]+" ");
}
private static void ordena_asc(int[] A, int L2, int L1) {
    for(int i=L1; i<=L2-1; i++) {
        int p=i;
        for(int k=i+1; k<=L2; k++) {
            if(A[k]>A[p]) {
                p=k;
            }
        }
        int aux=A[p];
        A[p]=A[i];
        A[i]=aux;
    }
}
private static void leerVector(int[] A, int n) {
    Scanner leer = new Scanner(System.in);
    for(int i=1; i<=n; i++) {
        A[i]= leer.nextInt();
    }
}
}

```

```

private static void ordena_des(int[] A, int L2, int L1) {
    for(int i=L1; i<=L2-1; i++) {
        int p=i;
        for(int k=i+1; k<=L2; k++) {
            if(A[k]<A[p]) {
                p=k;
            }
        }
        int aux=A[p];
        A[p]=A[i];
        A[i]=aux;}
    }
}

```

15

7

4

5

Inicio del programa:

4

-3

22

1

15

0

-2

8

12

3

16

4

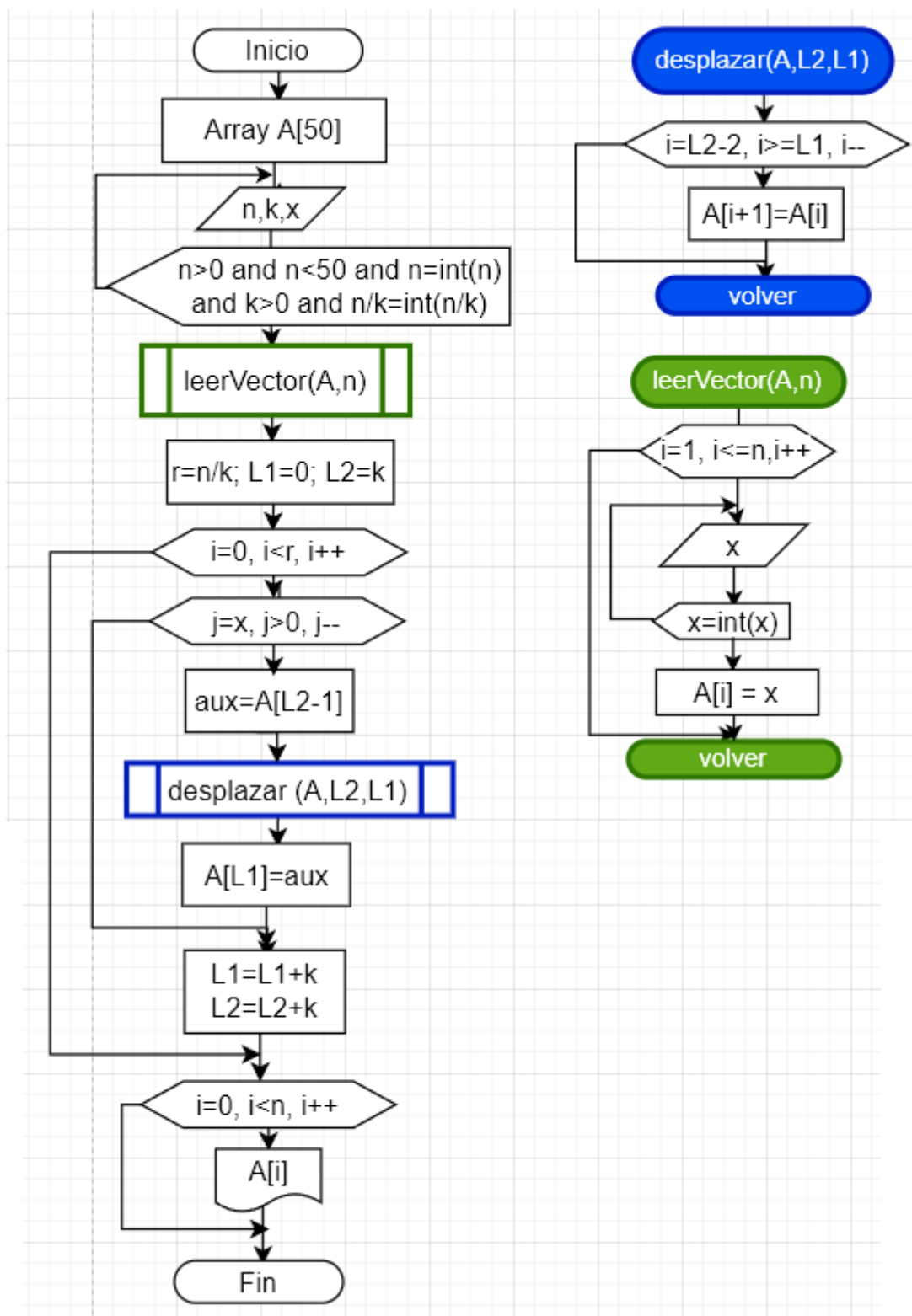
1

7

2

-3 1 4 15 22 12 8 3 0 -2 1 2 4 7 16

2.- Se tiene un vector V de tamaño n, rotar el vector cada k elementos x veces, k que divide exactamente a n.



```
//Ejercicio 2 division de vectores desplazar
Scanner leer = new Scanner(System.in);
int A[]=new int[50];
int n,aux,x;
int k;
do {
    n=leer.nextInt();
}while(n<=0 || n>50);
do {
    k=leer.nextInt();
}while(k<=0 || (n%k)!=0);
System.out.println("Ingrese can de rotacion: ");
do {
    x=leer.nextInt();
}while(x<=0);
System.out.println("Digite los numeros: ");
leerVector(A,n);
for(int i=0; i<n;i++) {
    System.out.print(A[i]+" ");
}
int r=n/k; int L1=0; int L2=k;
for(int i=0; i<r; i++) {
    for (int j=x; j>0; j--) {
        aux = A[L2-1];
        desplazar(A, L2, L1);
        A[L1] = aux;
    }
    L1=L1+k;
    L2=L2+k;
}
System.out.println();
for(int i=0; i<n;i++) {
    System.out.print(A[i]+" ");
}
}

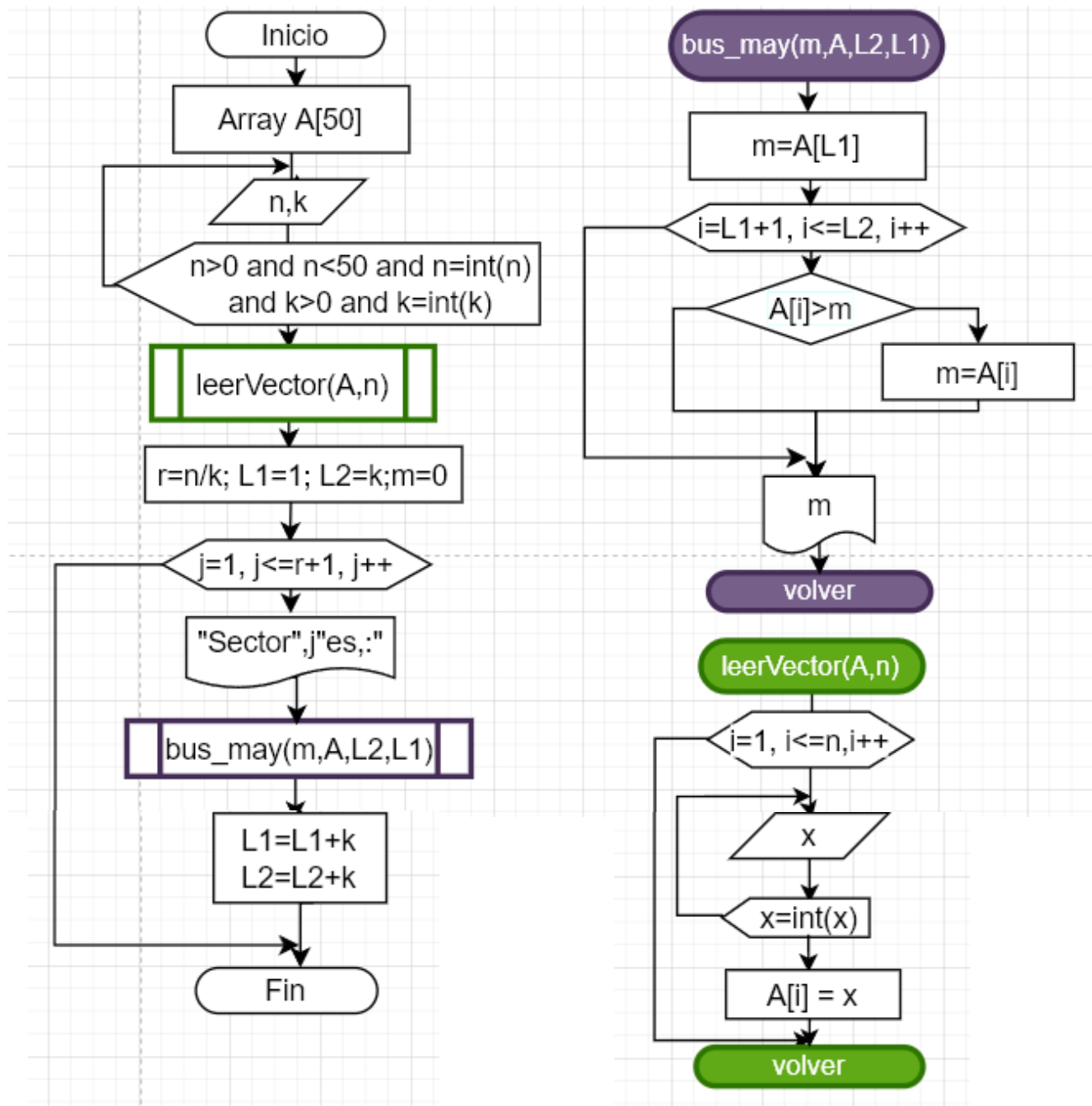
private static void desplazar(int[] A, int L2, int L1) {
    for(int i=L2-2; i>=L1; i--) {
        A[i+1] = A[i];
    }
}

private static void leerVector(int[] A, int n) {
    Scanner leer = new Scanner(System.in);
    for(int i=0; i<n; i++) {
        A[i] = leer.nextInt();
    }
}
```

```
12
4
Ingrese can de rotacion:
2
Digite los numeros:
10
6
12
1
9
2
8
11
16
3
5
7
10 6 12 1 9 2 8 11 16 3 5 7
12 1 10 6 8 11 9 2 5 7 16 3
```

3.- Realizar el ejemplo 1 de este documento con cualquier K, K no necesariamente divide exactamente a N. Puede existir un sector con menos elementos que k.

Para N=18 y K=5, tendremos 3 sectores de 5 elementos, y el último incompleto de 3 elementos



```

// ejercicio 3 encontrar el mayor de vectores divididos
Scanner leer = new Scanner(System.in);
int A[]=new int[50];
int n, k;
do {
    n=leer.nextInt();
}while(n<=0);
do {
    k=leer.nextInt();
}while(k<=0);
leerVector(A,n);
int r=n/k; int L1=1; int L2=k; int m=0;
for(int j=1; j<=r+1; j++) {
    System.out.print("Sector "+j+" es ");
    bus_may(m,A, L2,L1);
    L1=L1+k;
    L2=L2+k;}
}
private static void leerVector(int[] A, int n) {
    Scanner leer = new Scanner(System.in);
    for(int i=1; i<=n; i++) {
        A[i]= leer.nextInt();
    }
}
private static void bus_may(int m, int[] A, int L2, int L1) {
    m = A[L1];
    for(int i=L1+1; i<=L2; i++) {
        if(A[i]>m) {
            m=A[i];}
    }
    System.out.print(m+" ");
    System.out.println();
}
}

```

```

18
5
5
-8
9
12
0
4
-1
9
25
13
6
8
15
7
2
5
11
4
Sector 1 es 12
Sector 2 es 25
Sector 3 es 15
Sector 4 es 11

```